

HTML Forms, Validation & Multimedia

Comprehensive technical reference for web developers

1. HTML Input Types

HTML5 provides specialized input types that enable native user interface controls, mobile-optimized virtual keyboards, and basic browser-level validation.

Input Type	Description	Key Feature / UX Impact
<code>text</code>	Standard single-line text entry.	Default fallback input.
<code>email</code>	Validates structure matching <code>user@domain</code> .	Triggers <code>@</code> keyboard layout on mobile devices.
<code>password</code>	Masks entered characters with dots/asterisks.	Prevents shoulder surfing security leaks.
<code>number</code>	Accepts numerical values.	Supports <code>min</code> , <code>max</code> , and <code>step</code> controls.
<code>url</code>	Requires valid absolute URL (e.g. <code>https://...</code>).	Triggers <code>.com</code> and <code>/</code> mobile keyboard shortcuts.
<code>tel</code>	Designed for telephone numbers.	Opens numeric dial pad on mobile.
<code>date</code> / <code>time</code>	Date and time picker interfaces.	Native date-picker widget provided by browser.
<code>checkbox</code> / <code>radio</code>	Toggle choices and group selections.	Radio inputs require shared <code>name</code> attribute.

2. Client-Side vs. Server-Side Validation

Client-Side Validation (Browser Level)

Executed before data transmission. Provides fast user feedback and enhances UX without server round-trips.

- `required`: Ensures the input field is non-empty upon form submission.
- `minlength` & `maxlength`: Restricts the character count allowed in text fields.
- `min` & `max`: Restricts upper and lower numeric or date bounds.
- `pattern`: Enforces value matching against a JavaScript Regular Expression (Regex).

CSS Pseudo-Classes for Styling Validation:

```
/* Style required fields */
input:required {
  border-right: 3px solid #f59e0b;
```

```

}

/* Style invalid fields when constraint fails */
input:invalid {
  border: 2px solid #ef4444;
  background-color: #fef2f2;
}

/* Style valid input */
input:valid {
  border: 1px solid #10b981;
}

```

Security Rule: Client-side validation can easily be bypassed by disabling JavaScript, modifying the DOM, or using HTTP clients (e.g., Postman, Curl). It is strictly for user experience, not security.

Server-Side Validation (Back-End Level)

Evaluates submitted data on the server application layer (Node.js, Python, PHP, etc.) before writing to a database or executing business operations.

- **Mandatory Security Layer:** Protects against SQL Injection, Cross-Site Scripting (XSS), Parameter Tampering, and malicious payloads.
- **Data Integrity:** Re-checks types, lengths, uniqueness, and business requirements regardless of client constraints.

3. Form Submission Methods: GET vs. POST

Choosing the correct HTTP method for form submission is crucial for security and application behavior.

Feature	GET Method	POST Method
Data Location	Appended to URL query string (<code>?key=val</code>).	Included inside HTTP Request Body.
Data Visibility	Visible in address bar, logs, and browser history.	Hidden from address bar and server logs.
Capacity / Size	Limited by maximum URL length (~2,048 chars).	No inherent size limit (suitable for file uploads).
Caching & Bookmarks	Can be cached and bookmarked.	Cannot be cached or bookmarked.
Primary Use Case	Read-only actions (Search, filtering, data lookup).	State-altering actions (Registration, login, file uploads).

Common GET & POST Issues

- **GET Security Leak:** Passing passwords or tokens via GET exposes credentials in server logs and browser histories.
- **Form Resubmission Warning (POST):** Navigating back to a POST page displays a browser reload warning ("Confirm Form Resubmission"). Prevent this using the *Post/Redirect/Get (PRG)* architectural pattern.

4. Audio & Video Attributes in HTML5

HTML5 provides built-in media players without requiring third-party plugins.

`<video>` Tag Attributes

Used to embed video content in web documents.

- `src` : Path or URL to the video file.
- `controls` : Renders browser native play, pause, volume, and fullscreen controls.
- `autoplay` : Begins playback automatically (modern browsers require `muted` for autoplay to work).
- `muted` : Silences audio track by default.
- `loop` : Automatically restarts playback when finished.
- `poster` : Specifies an image displayed before video playback begins.
- `width` / `height` : Sets visual dimension boundaries in pixels.

`<audio>` Tag Attributes

Used to embed sound files or audio streams.

- `src` : Path or URL to the audio source.
- `controls` : Renders native audio play/pause controls.
- `autoplay` : Starts audio playback automatically upon page load.
- `muted` : Mutes initial audio playback.
- `loop` : Causes the audio file to repeat continuously.
- `preload` : Hints to the browser how audio should be buffered (`none` , `metadata` , or `auto`).